```
        READFILE "Test.txt", TextString
        OUTPUT TextString
    ENDWHILE
    CLOSEFILE "Test.txt"
```

TASK 13.08

1 Write pseudocode to save the array data from Task 13.06 to a text file.

2 Write pseudocode to read the values stored in the text file back into the board array.

13.07 Abstract Data Types (ADTs)

An **Abstract Data Type** is a collection of data and a set of associated operations:

- create a new instance of the data structure
- find an element in the data structure
- insert a new element into the data structure
- delete an element from the data structure
- access all elements stored in the data structure in a systematic manner.

KEY TERM

Abstract Data Type: a collection of data with associated operations

The remainder of this chapter describes the following ADTs: stack, queue and linked list. It also demonstrates how they can be implemented from arrays.

In the following ADTs data items are represented as a single character, but this would normally be a set of data, possibly stored as fields in a record.

13.08 Stacks

What are the features of a stack in the real world? To make a stack, we pile items on top of each other. The item that is accessible is the one on top of the stack. If we try to find an item in the stack and take it out, we are likely to cause the pile of items to collapse.

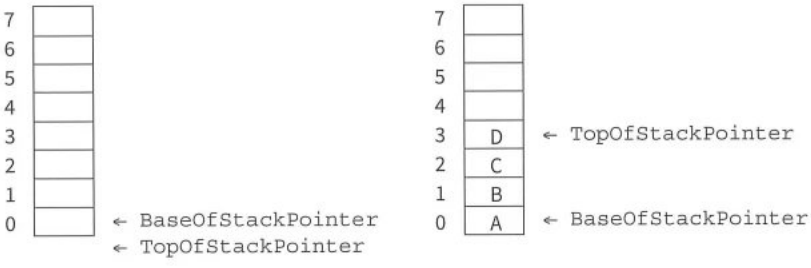


Figure 13.07 An empty stack (left) and a stack with four items pushed (right)

Figure 13.07 shows how we can represent a stack when we have added four items in this order: A, B, C, D. Note that the slots are shown numbered from the bottom as this feels more natural.

The `BaseOfStackPointer` will always point to the first slot in the stack. The `TopOfStackPointer` will point to the last element pushed (added) onto the stack. When an element is popped (removed) from the stack, the `TopOfStackPointer` will decrease to point to the element now at the top of the stack. When the stack is empty, `TopOfStackPointer` will have the value –1.

To implement this stack using a 1D array, we write:

```
DECLARE Stack:ARRAY[0:7] OF CHAR
```

TASK 13.09

1 Draw a diagram to show the contents of the stack shown in Figure 13.07 after "E" has been pushed onto the stack.

2 Draw a diagram to show the contents of the stack shown in Figure 13.07 after one item has been popped off the stack.

13.09 Queues

What are the features of a queue in the real world? When people form a queue, they join the queue at the end. People leave the queue from the front of the queue. If it is an orderly queue, no-one pushes in between and people don't leave the queue from any other position.

Figure 13.08 shows how we can represent a queue when five items have joined the queue in this order: A, B, C, D, E.

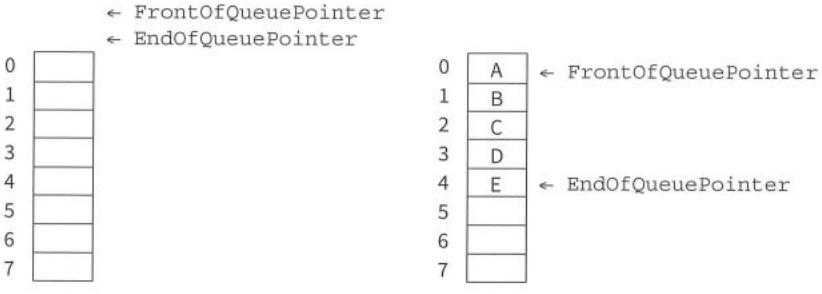


Figure 13.08 An empty queue (left) and a queue after 5 items have joined (right)

To implement a queue using an array, we can assume that the front of the queue is at position 0. When the queue is empty, the `EndOfQueuePointer` will have the value –1. When one value joins the queue, the `EndOfQueuePointer` will be incremented before adding the value to the array element where the pointer is pointing to. When the item at the front of the queue leaves, we need to move all the other items one slot forward and adjust `EndOfQueuePointer`.

TASK 13.10

1 Draw a diagram to show the contents of the queue after "F" has joined the non-empty queue shown in Figure 13.08.

2 Draw a diagram to show the contents of the queue after one item has left the non-empty queue shown in Figure 13.08.

This method involves a lot of moving of data. A more efficient way to make use of the slots is the concept of a 'circular' queue. Pointers show where the front and end of the queue are. Eventually the queue will 'wrap around' to the beginning. Figure 13.09 shows a circular queue after 11 items have joined and five items have left the queue.

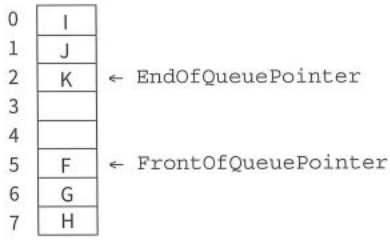


Figure 13.09 A circular queue

13.10 Linked lists

In Section 13.03 we used an array as a linear list. In a linear list, the list items are stored in consecutive locations. This is not always appropriate. Another method is to store an individual list item in whatever location is available and link the individual item into an ordered sequence using pointers.

An element of a list is called a node. A **node** can consist of several data items and a **pointer**, which is a variable that stores the address of the node it points to.

A pointer that does not point at anything is called a **null pointer.** It is usually represented by ∅. A variable that stores the address of the first element is called a **start pointer.**

KEY TERMS

Node: an element of a list

Pointer: a variable that stores the address of the node it points to

Null pointer: a pointer that does not point at anything

Start pointer: a variable that stores the address of the first element of a linked list

In Figure 13.10, the data value in the node box represents the key field of that node. There are likely to be many data items associated with each node. The arrows represent the pointers. It does not show at which address a node is stored, so the diagram does not give the value of the pointer, only where it conceptually links to.